

Advanced concurrency

COMPLETE LEARNING GUIDE

What You'll Learn

Beyond the basics of `Thread` and `synchronized`, `java.util.concurrent` provides a rich toolkit of higher-level building blocks that make concurrent code both easier to write correctly and more scalable — this is the toolkit real production Java systems actually use. This guide covers the `Executor` framework and thread pools in depth, `Future` and the more powerful `CompletableFuture` for composing asynchronous operations without callback nesting, concurrent collections (`ConcurrentHashMap`, `CopyOnWriteArrayList`) that are safe under concurrent access without external synchronization, explicit locks (`ReentrantLock`) and atomic variables (`AtomicInteger`) as alternatives to `synchronized`, coordination utilities (`CountDownLatch`, `CyclicBarrier`), and a concrete look at how deadlocks happen and how to prevent them. This is dense, interview-relevant material — expect questions on the difference between `synchronized` and `ReentrantLock`, and on how to compose multiple async operations with `CompletableFuture`.

1. Thread Pools and the Executor Framework

`Executors` provides factory methods for common thread pool configurations: `newFixedThreadPool(n)` keeps exactly `n` threads alive, queuing excess tasks; `newCachedThreadPool()` creates threads on demand and reuses idle ones, growing unbounded under sustained load (risky for unbounded workloads); `newSingleThreadExecutor()` serializes all tasks through one thread, guaranteeing sequential execution. `submit()` returns a `Future` representing the pending result, while `execute()` (inherited from `Executor`) is fire-and-forget with no result tracking. Always call `shutdown()` (graceful, finishes queued tasks) or `shutdownNow()` (attempts to cancel in-flight tasks) when a pool is no longer needed — an `ExecutorService` that's never shut down keeps its threads alive and can prevent the JVM from exiting.

JAVA

```
ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Integer> future = pool.submit(() -> {
    Thread.sleep(100);
    return 42;
});
Integer result = future.get(); // blocks until the task completes
pool.shutdown();
```

2. CompletableFuture: Composing Async Operations

Future only supports blocking `get()` to retrieve a result — it has no way to chain a follow-up action or combine multiple futures without manually blocking each one. `CompletableFuture` (Java 8+) solves this by supporting non-blocking composition: `thenApply()` transforms a result once available, `thenCompose()` chains a dependent async operation (flattening nested futures, analogous to `flatMap`), and `thenCombine()` merges two independent futures' results once both complete. This eliminates deeply nested callback code while still running work asynchronously on a thread pool (the common `ForkJoinPool.commonPool()` by default, or a custom `Executor` passed explicitly).

JAVA

```
CompletableFuture<Integer> fetchUserAge = CompletableFuture.supplyAsync(() ->
    fetchAge(userId));

CompletableFuture<String> greeting = fetchUserAge
    .thenApply(age -> age >= 18 ? "adult" : "minor")
    .thenCompose(category -> fetchGreetingFor(category)); // chains another
    async call

greeting.thenAccept(System.out::println); // non-blocking callback on
completion
```

3. Concurrent Collections

`ConcurrentHashMap` provides thread-safe `get/put` without locking the entire map for every operation — it internally partitions locking at a fine-grained level (historically per-segment, now per-bucket since Java 8), so multiple threads can read and write different keys simultaneously without blocking each other, making it dramatically faster under contention than a manually synchronized `HashMap` or the legacy `Hashtable`. `CopyOnWriteArrayList` is optimized for read-heavy, write-rare scenarios: every mutation (add/remove) copies the entire underlying array, making writes expensive but reads (including iteration) require no locking at all and are never affected by concurrent modification — ideal for something like a list of event listeners that's iterated constantly but modified rarely.

- `ConcurrentHashMap` — fine-grained locking, high-throughput reads and writes; never throws `ConcurrentModificationException` during iteration.
- `CopyOnWriteArrayList` — read operations are lock-free and always safe; writes are $O(n)$ due to full-array copying — use only when reads vastly outnumber writes.

- Both are found in `java.util.concurrent`, distinct from the plain (unsynchronized) `java.util` collections.

4. ReentrantLock and Atomic Variables

`ReentrantLock` offers everything `synchronized` does, plus features `synchronized` cannot express: `tryLock()` (attempt to acquire without blocking indefinitely, optionally with a timeout), interruptible lock acquisition, and fairness ordering (constructing with `new ReentrantLock(true)` grants the lock to the longest-waiting thread first, reducing thread starvation). Unlike `synchronized`, a `ReentrantLock` must be released manually in a `finally` block — forgetting to `unlock()` leaves the lock permanently held, a bug `synchronized`'s automatic release makes structurally impossible. Atomic classes (`AtomicInteger`, `AtomicLong`, `AtomicReference`) provide lock-free, hardware-level compare-and-swap operations for single-variable updates like counters, avoiding synchronization overhead entirely for the common case of one shared numeric field.

JAVA

```
private final ReentrantLock lock = new ReentrantLock();

public void criticalSection() {
    lock.lock();
    try {
        // protected code
    } finally {
        lock.unlock(); // MUST be in finally, or a lock leaks on exception
    }
}

// Atomic alternative to synchronized for a simple counter
private final AtomicInteger counter = new AtomicInteger(0);
public void increment() { counter.incrementAndGet(); } // lock-free, thread-safe
```

5. Coordination Utilities: CountdownLatch and CyclicBarrier

`CountDownLatch` lets one or more threads wait until a set of operations happening in other threads completes: initialized with a count, each worker calls `countDown()` when finished, and any thread calling `await()` blocks until the count reaches zero — commonly used to wait for several parallel initialization tasks to finish before proceeding, and it's single-use (cannot be reset once it reaches zero). `CyclicBarrier` is similar but reusable and inverted in purpose: it makes a fixed number of threads all wait for each other at a common point before any of them proceeds, useful for phased, lock-step parallel algorithms where every worker must finish phase N before any worker starts phase N+1.

6. Deadlocks: How They Happen and How to Avoid Them

A deadlock occurs when two or more threads each hold a lock the other needs and neither can proceed — the classic case is Thread A locking resource 1 then waiting for resource 2, while Thread B simultaneously locks resource 2 then waits for resource 1, so both threads block forever with no way out. The standard prevention strategy is lock ordering: always acquire multiple locks in the same globally consistent order across every code path that needs more than one, which makes the circular-wait condition required for deadlock structurally impossible. `tryLock()` with a timeout (available on `ReentrantLock` but not `synchronized`) offers a second mitigation — a thread that can't acquire a lock within a bounded time can back off and retry instead of blocking forever.

JAVA

```
// Deadlock risk: inconsistent lock ordering between two methods
void transferAB() { synchronized (lockA) { synchronized (lockB) { /* ... */ } } }
void transferBA() { synchronized (lockB) { synchronized (lockA) { /* ...
*/ } } } // WRONG order

// Fix: always acquire in the same global order (e.g. by object identity hash)
void transfer(Object first, Object second) {
    synchronized (first) {
        synchronized (second) { /* ... */ }
    }
}
```

Common Pitfalls

- Never calling `shutdown()` on an `ExecutorService`, keeping its threads alive indefinitely and preventing clean JVM shutdown.
- Calling `future.get()` with no timeout, blocking forever if the submitted task hangs — prefer `get(timeout, unit)` in production code.
- Acquiring locks in inconsistent order across different methods, creating a classic circular-wait deadlock under concurrent access.
- Forgetting to release a `ReentrantLock` in a `finally` block, permanently leaking the lock if an exception occurs in the critical section.
- Using `CopyOnWriteArrayList` for a write-heavy workload, incurring an $O(n)$ full-array copy on every single mutation.
- Reusing a `CountDownLatch` after it reaches zero, not realizing it's single-use — `CyclicBarrier` is the reusable equivalent.
- Mixing raw `synchronized` blocks and `ReentrantLock` on what's meant to be the same critical section, providing no actual mutual exclusion since they're different locking mechanisms.

Key Takeaways

- Prefer a pooled `ExecutorService` over raw `Thread` objects, and always shut it down when finished.
- `CompletableFuture` supports non-blocking composition (`thenApply/thenCompose/thenCombine`) where a plain `Future` only supports blocking `get()`.
- `ConcurrentHashMap` and `CopyOnWriteArrayList` provide thread safety without external synchronization, each optimized for a different read/write balance.
- `ReentrantLock` adds `tryLock`, timeouts, and fairness over `synchronized`, but requires manual `unlock()` in a `finally` block.
- Atomic classes (`AtomicInteger`, etc.) give lock-free thread safety for single-variable updates via compare-and-swap.
- Deadlocks require a circular wait between locks — consistent lock ordering across the entire codebase prevents it structurally.

Learn More

Oracle Java Tutorials — High Level Concurrency Objects

<https://docs.oracle.com/javase/tutorial/essential/concurrency/highlevel.html>

Oracle Java SE Docs — `CompletableFuture`

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/CompletableFuture.html>

Baeldung — Guide to `java.util.concurrent.Locks`

<https://www.baeldung.com/java-concurrent-locks>